JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Enum Class ChronoUnit

java.lang.Object
    java.lang.Enum<ChronoUnit>
        java.time.temporal.ChronoUnit

All Implemented Interfaces:

Serializable, Comparable<ChronoUnit>, Constable, TemporalUnit

---

```
public enum ChronoUnit
extends Enum<ChronoUnit>
implements TemporalUnit
```

A standard set of date periods units.

This set of units provide unit-based access to manipulate a date, time or date-time. The standard set of units can be extended by implementing TemporalUnit.

These units are intended to be applicable in multiple calendar systems. For example, most non-ISO calendar systems define units of years, months and days, just with slightly different rules. The documentation of each unit explains how it operates.

Implementation Requirements:

This is a final, immutable and thread-safe enum.

Since:

1.8

Nested Class Summary

Nested classes/interfaces declared in class Enum
Enum.EnumDesc<E>

Enum Constant Summary

Enum Constants

Enum Constant	Description
CENTURIES	Unit that represents the concept of a century.
DAYS	Unit that represents the concept of a day.

DECADES	Unit that represents the concept of a decade.
ERAS	Unit that represents the concept of an era.
FOREVER	Artificial unit that represents the concept of forever.
HALF_DAYS	Unit that represents the concept of half a day, as used in AM/PM.
HOURS	Unit that represents the concept of an hour.
MICROS	Unit that represents the concept of a microsecond.
MILLENNIA	Unit that represents the concept of a millennium.
MILLIS	Unit that represents the concept of a millisecond.
MINUTES	Unit that represents the concept of a minute.
MONTHS	Unit that represents the concept of a month.
NANOS	Unit that represents the concept of a nanosecond, the smallest supported unit of time.
SECONDS	Unit that represents the concept of a second.
WEEKS	Unit that represents the concept of a week.
YEARS	Unit that represents the concept of a year.

Method Summary

Modifier and Type	Method	Description
<R extends Temporal> R	**addTo**(R temporal, long amount)	Returns a copy of the specified temporal object with the specified period added.
long	**between**(Temporal temporal1Inclu Temporal temporal2Exclus	Calculates the amount of time between two temporal objects.
Duration	**getDuration**()	Gets the estimated duration of this unit in the ISO calendar system.
boolean	**isDateBased**()	Checks if this unit is a date unit.
boolean	**isDurationEstimated**()	Checks if the duration of the unit is an estimate.
boolean	**isTimeBased**()	Checks if this unit is a time unit.
String	**toString**()	Returns the name of this enum constant, as contained in the declaration.
static ChronoUnit	**valueOf**(String name)	Returns the enum constant of this class with the specified name.
static ChronoUnit[]	**values**()	Returns an array containing the constants of this enum class, in the order they are declared.

Methods declared in class Enum

clone, compareTo, describeConstable, equals, finalize, getDeclaringClass, hashCode, name, ordinal, toString, valueOf

Methods declared in class Object

getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface TemporalUnit

`isSupportedBy`

Enum Constant Details

NANOS

`public static final ChronoUnit NANOS`

Unit that represents the concept of a nanosecond, the smallest supported unit of time. For the ISO calendar system, it is equal to the 1,000,000,000th part of the second unit.

MICROS

`public static final ChronoUnit MICROS`

Unit that represents the concept of a microsecond. For the ISO calendar system, it is equal to the 1,000,000th part of the second unit.

MILLIS

`public static final ChronoUnit MILLIS`

Unit that represents the concept of a millisecond. For the ISO calendar system, it is equal to the 1000th part of the second unit.

SECONDS

`public static final ChronoUnit SECONDS`

Unit that represents the concept of a second. For the ISO calendar system, it is equal to the second in the SI system of units, except around a leap-second.

MINUTES

`public static final ChronoUnit MINUTES`

Unit that represents the concept of a minute. For the ISO calendar system, it is equal to 60 seconds.

HOURS

`public static final ChronoUnit HOURS`

Unit that represents the concept of an hour. For the ISO calendar system, it is equal to 60 minutes.

HALF_DAYS

`public static final ChronoUnit HALF_DAYS`

Unit that represents the concept of half a day, as used in AM/PM. For the ISO calendar system, it is equal to 12 hours.

DAYS

`public static final ChronoUnit DAYS`

Unit that represents the concept of a day. For the ISO calendar system, it is the standard day from midnight to midnight. The estimated duration of a day is `24 Hours`.

When used with other calendar systems it must correspond to the day defined by the rising and setting of the Sun on Earth. It is not required that days begin at midnight - when converting between calendar systems, the date should be equivalent at midday.

WEEKS

`public static final ChronoUnit WEEKS`

Unit that represents the concept of a week. For the ISO calendar system, it is equal to 7 days.

When used with other calendar systems it must correspond to an integral number of days.

MONTHS

`public static final ChronoUnit MONTHS`

Unit that represents the concept of a month. For the ISO calendar system, the length of the month varies by month-of-year. The estimated duration of a month is one twelfth of `365.2425 Days`.

When used with other calendar systems it must correspond to an integral number of days.

YEARS

`public static final ChronoUnit YEARS`

Unit that represents the concept of a year. For the ISO calendar system, it is equal to 12 months. The estimated duration of a year is `365.2425 Days`.

When used with other calendar systems it must correspond to an integral number of days or months roughly equal to a year defined by the passage of the Earth around the Sun.

DECADES

`public static final ChronoUnit DECADES`

Unit that represents the concept of a decade. For the ISO calendar system, it is equal to 10 years.

When used with other calendar systems it must correspond to an integral number of days and is normally an integral number of years.

CENTURIES

`public static final ChronoUnit CENTURIES`

Unit that represents the concept of a century. For the ISO calendar system, it is equal to 100 years.

When used with other calendar systems it must correspond to an integral number of days and is normally an integral number of years.

MILLENNIA

`public static final ChronoUnit MILLENNIA`

Unit that represents the concept of a millennium. For the ISO calendar system, it is equal to 1000 years.

When used with other calendar systems it must correspond to an integral number of days and is normally an integral number of years.

ERAS

`public static final ChronoUnit ERAS`

Unit that represents the concept of an era. The ISO calendar system doesn't have eras thus it is impossible to add an era to a date or date-time. The estimated duration of the era is artificially defined as `1,000,000,000 Years`.

When used with other calendar systems there are no restrictions on the unit.

FOREVER

`public static final ChronoUnit FOREVER`

Artificial unit that represents the concept of forever. This is primarily used with `TemporalField` to represent unbounded fields such as the year or era. The estimated duration of this unit is artificially defined as `Duration.MAX`.

Method Details

values

```
public static ChronoUnit[] values()
```

Returns an array containing the constants of this enum class, in the order they are declared.

Returns:

an array containing the constants of this enum class, in the order they are declared

valueOf

```
public static ChronoUnit valueOf(String name)
```

Returns the enum constant of this class with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this class. (Extraneous whitespace characters are not permitted.)

Parameters:

`name` - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

`IllegalArgumentException` - if this enum class has no constant with the specified name

`NullPointerException` - if the argument is null

getDuration

```
public Duration getDuration()
```

Gets the estimated duration of this unit in the ISO calendar system.

All of the units in this class have an estimated duration. Days vary due to daylight saving time, while months have different lengths.

Specified by:

`getDuration` in interface `TemporalUnit`

Returns:

the estimated duration of this unit, not null

isDurationEstimated

`public boolean isDurationEstimated()`

Checks if the duration of the unit is an estimate.

All time units in this class are considered to be accurate, while all date units in this class are considered to be estimated.

This definition ignores leap seconds, but considers that Days vary due to daylight saving time and months have different lengths.

Specified by:

`isDurationEstimated` in interface `TemporalUnit`

Returns:

true if the duration is estimated, false if accurate

isDateBased

`public boolean isDateBased()`

Checks if this unit is a date unit.

All units from days to eras inclusive are date-based. Time-based units and `FOREVER` return false.

Specified by:

`isDateBased` in interface `TemporalUnit`

Returns:

true if a date unit, false if a time unit

isTimeBased

`public boolean isTimeBased()`

Checks if this unit is a time unit.

All units from nanos to half-days inclusive are time-based. Date-based units and `FOREVER` return false.

Specified by:

`isTimeBased` in interface `TemporalUnit`

Returns:

true if a time unit, false if a date unit

addTo

```
public <R extends Temporal> R addTo(R temporal,
                                    long amount)
```

Description copied from interface: `TemporalUnit`

Returns a copy of the specified temporal object with the specified period added.

The period added is a multiple of this unit. For example, this method could be used to add "3 days" to a date by calling this method on the instance representing "days", passing the date and the period "3". The period to be added may be negative, which is equivalent to subtraction.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.plus(long, TemporalUnit)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisUnit.addTo(temporal, amount);
temporal = temporal.plus(amount, thisUnit);
```

It is recommended to use the second approach, `plus(amount, thisUnit)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the units available in `ChronoUnit` or the fields available in `ChronoField`. If the unit is not supported an `UnsupportedTemporalTypeException` must be thrown.

Implementations must not alter the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

Specified by:

`addTo` in interface `TemporalUnit`

Type Parameters:

`R` - the type of the Temporal object

Parameters:

`temporal` - the temporal object to adjust, not null

`amount` - the amount of this unit to add, positive or negative

Returns:

the adjusted temporal object, not null

between

```
public long between(Temporal temporal1Inclusive,
                    Temporal temporal2Exclusive)
```

Description copied from interface: `TemporalUnit`

Calculates the amount of time between two temporal objects.

This calculates the amount in terms of this unit. The start and end points are supplied as temporal objects and must be of compatible types. The implementation will convert the second type to be an instance of the first type before the calculating the amount. The result will be negative if the end is before the start. For example, the amount in hours between two temporal objects can be calculated using `HOURS.between(startTime, endTime)`.

The calculation returns a whole number, representing the number of complete units between the two temporals. If there are smaller unit fields, their values are considered when determining the final whole number. For example, the amount in hours between the times 11:30 and 13:29 will only be one hour as it is one minute short of two hours, or the amount in months between the dates 2024-09-29 and 2025-02-28 (the last day in February) will be 4 months as it is one day short of 5 months.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.until(Temporal, TemporalUnit)`:

```
// these two lines are equivalent
between = thisUnit.between(start, end);
between = start.until(end, thisUnit);
```

The choice should be made based on which makes the code more readable.

For example, this method allows the number of days between two dates to be calculated:

```
long daysBetween = DAYS.between(start, end);
// or alternatively
long daysBetween = start.until(end, DAYS);
```

Implementations should perform any queries or calculations using the units available in `ChronoUnit` or the fields available in `ChronoField`. If the unit is not supported an `UnsupportedTemporalTypeException` must be thrown. Implementations must not alter the specified temporal objects.

Specified by:

between in interface TemporalUnit

Parameters:

temporal1Inclusive - the base temporal object, not null

temporal2Exclusive - the other temporal object, exclusive, not null

Returns:

the amount of time between temporal1Inclusive and temporal2Exclusive in terms of this unit; positive if temporal2Exclusive is later than temporal1Inclusive, negative if earlier

toString

```
public String toString()
```

Description copied from class: Enum

Returns the name of this enum constant, as contained in the declaration. This method may be overridden, though it typically isn't necessary or desirable. An enum class should override this method when a more "programmer-friendly" string form exists.

Specified by:

toString in interface TemporalUnit

Overrides:

toString in class Enum<ChronoUnit>

Returns:

the name of this enum constant

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.